Open-Source Audit Report

Software Chosen: MySQL
Student Name: Varun Kumar Sinha
Registration Number: 24BCE10999
Course: OPEN-SOURCE Software
Slot - B22
Date of Submission: 30-03-2026

1. Introduction

Most of the software that powers the modern world – the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones – was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project is an opportunity to stop and reflect on that reality. It is incredibly easy to treat the digital infrastructure we interact with daily as a series of black boxes, taking for granted that it simply works. Yet, nearly every major digital platform sits heavily upon open foundations. Understanding the philosophy behind this ecosystem – why these tools were built, who maintains them, and what specific freedoms their licenses guarantee – is not just an academic footnote; it is a critical part of understanding modern technology itself.

To explore these concepts practically, I have choose to conduct a structured audit of MySQL. Relational databases act as the fundamental memory banks of the internet, and MySQL stands out as the crucial open-source bridge that allowed dynamic, data-driven websites to flourish globally. Through this capstone project, the objective is to look past the basic functionality of the database and deeply examine the ecosystem that sustains it.

Over the following sections, this report will detail the historical problem MySQL was originally created to solve and analyze the specific legal boundaries of its GPLv2 license. I have also documented the practical installation and configuration footprint of MySQL within a Linux operating system. Finally, the audit will map the broader FOSS ecosystem that MySQL relies upon and conclude with a critical, objective comparison against a proprietary enterprise alternative to evaluate its ongoing relevance in the real world.

2. Chosen Software: MySQL

PART A — Origin and Philosophy

A1. The problem this software was created to solve

The Pre-Existing Landscape

We all know that the mid-1990s the decade which MySQL was born had a different computing world. Relational database managements systems (RDBMS) back then were the properties of only gigantic enterprise establishments. If you wished for a dependable, ACID objection table, you would have to purchase one in form of a proprietary software either from Oracle or from Microsoft SQL Server. These alternatives were just too expensive, and not to mention needed high-end server hardware and forced you to get specialized administers.

Lightweight tools were also at the other end of the spectrum. mSQL (Mini SQL) was one such tool, but it was extremely limited. It did not have basic features needed for speed and for handling complex data relationships, like strong indexing. It was okay for hobbyists, but totally unsuitable for the needs of the fast growing World Wide Web as it was fundamentally broken.

The Catalyst for Creation

It was mSQL's inadequacy that led its founders, Michael "Monty" Widenius and David Axmark to develop something specifically for the task at hand. Their task involved linking tables to their web applications over the net. However, since this did not work efficiently due to the poor performance and rigidity of mQSL, they had no option but to resort into utilizing expensive proprietary systems until Widenius found an alternative approach by designing a low level very efficient database engine UNIREG which he later modified to be able to understand SQL (Structured Query Language) so as to avoid these high costs.

Why Share It?

They chose to do this and used the strategy of building it openly and sharing it for free, rather than keeping it proprietary for a very high-level reason: they needed the software to be battle-tested. So by releasing it to this nascent web development community, they effectively crowdsourced their QA (Quality Assurance) and debugging. The internet was growing at an exponential rate and there simply weren't too many options for good, low-cost and reliable databases. In essence, people went with what they knew would work. MySQL filled the void because it

was easy to use, reliable; designed in a way which made easy integration with PHP and Apache. This solved a huge problem of accessibility; since MySQL meant that a student working alone could pretty much have the same powerful database capability as some rich company.

A2. The license — what it actually says

The License: What It Actually Says

In the license, MySQL has actually presented a very fabulous case study in open-source software legality because it uses a dual-model licensing strategy. The version in the open-source community is controlled by the GNU General Public License, version 2 (GPLv2).

The Four Freedoms

The GPLv2 is designed in such a way to protect the four freedoms of the free software:

1. **Freedom 0:** We have the freedom to run the program as per our wish, for any purpose.
2. **Freedom 1:** We have the freedom to study how the program works, and also we can change it as per our needs so it does the computing as per our wish. Access to the source code is a precondition for this freedom.
3. **Freedom 2:** We have the freedom to redistribute the copies to our neighbor so you can help them.
4. **Freedom 3:** We also have the freedom to distribute copies of our own modified versions of the open-source software to others. By doing this, we can give the whole community a chance to get from our work or changes.

Commercial Modification and Distribution

Someone might think that can a company take MySQL, modify it and sell it without sharing their changes? Under the GPLv2 license, the answer is **no.** The GPL is a “copyleft” or viral license. If a company modifies the MySQL source code and distributes that new product to the public or clients, the license legally

compels them to release their modified source code under the exact same GPLv2 terms. They cannot lock it away as proprietary software.

However, MySQL found a solution for this, it bypassed this limitation for corporations through its **dual license**. If a company wishes to embed MySQL into a closed-source, proprietary application without releasing their code, they must purchase a commercial license from Oracle (who currently owns MySQL). This brilliant legal maneuver forces the software to remain free for the community while generating revenue from corporations.

GPL vs. MIT Licensing

The philosophical difference between GPL and MIT is very profound.

- **GPL** prioritizes the freedom of the software. It states that the code, and any code derived from it, must remain open forever.
- **MIT** prioritizes the freedom of the developer. It is a flexible license that allows anyone to take the code, modify it, and put it into a closed-source, proprietary, and profitable product without ever sharing their improvements.

Let’s say we were developing a web application – perhaps an automated system for detecting plant diseases using machine learning – and we want to ensure that no massive agricultural tech company could ever steal our code, close the source, and sell it for profit, then we should choose **GPL**. It will force anyone who builds upon our work to share their advancements with the world.

Free Beer vs. Free Speech

"Free as in free beer" implies **zero monetary cost**. Downloading the MySQL installer costs nothing.
"Free as in freedom" (free speech) refers to **liberty**. With MySQL, we have the liberty to download the actual source code written in C and C++, tear apart the InnoDB storage engine, rewrite how it handles memory allocation, and recompile it. We are granted the blueprints, not just the final product.

A3. The ethics of open source

Should All Software Be Open Source?

- **The Argument For:** Open Source software is essentially applied mathematics and logic. By putting restrictions on the access of the source code, it's similar to locking away the scientific discoveries or mathematical formulas. As when the software is universally open, security flaws, bugs are found and patched faster by a whole global community, due to this educational barriers are demolished and technological progress accelerates because developers do not have to constantly reinvent the wheel.
- **The Argument Against:** To create complex, user-friendly and enterprise-grade software one need a massive capital investment. We also don't need every software to be open-source, as if all software were mandatorily open source, the inner drive to use own finance to take massive risks on revolutionary new tools might just vanish. As proprietary software models guarantee that creators can recoup the costs of server infrastructure, employee salaries and research and development.

The Ethics of Corporate Exploitation

As the modern technology is expanding, it has brought renewed attention to the ethical dynamics, underlying the use of open-source software within commercial environments. Major companies like Google and Meta, rely extensively on foundational technologies such as Linux and MySQL, along with various other open-source libraries. These tools are often developed and maintained by communities where contributions are frequently voluntary and not always financially compensated.

This type of situation also raises important ethical questions regarding fairness and value distrbution. On one hand, it can be said that big companies benefit largely as compare to the small companies/communities from the resources that are collectively produced. In particular, when such big companies generate substantial revenue without consistently contributing back – either through funding, maintenance or active development – it creates a pov of imbalance. Some even describe this as a form of indirect exploitation, especially in cases where essential open-source projects depend on a limited number of members who may lack adequate institutional support.

On other hand, the relationship between big companies and open-source ecosystems is not entirely one-sided. Some big companies, like Red Hat as well as teams within Google, allocate significant resources toward the development and maintenance of open-source projects. This includes employing developers whose responsibilities contribute the most in widely used systems like the Linux kernel. These efforts, are not always directly proportional to the benefits received, nevertheless play a role in sustaining and advancing the open-source ecosystem.

Finally we can say, the interaction between large-scale companies and open-source communities reflects a complex and evolving balance. It cannot be fully categorized as purely exploitative or entirely collaborative. Instead, we can say it represents a hybrid model in which shared technological development coexists with uneven economic outcomes, leaving the ethical implications open to continued debate.

Standing on the Shoulders of Giants

In the context of software development, this idea tell us about the layered nature of modern computing systems, where contemporary applications represent only the uppermost level of a much larger and collectively constructed technological foundation. Most software developed today is not created alone; rather, it is built using an extensive base of existing tools, frameworks, and systems, many of which originate from open-source initiatives.

Like when developing a typical web application, the developer is not responsible for operating system it is using, configuring the web server architecture, implementing the database engine, or designing the programming language compiler. These are the components which are already available and been developed and refined over time by previous contributors who have made their work openly accessible.

This cumulative structure plays a significant role in shaping innovation within the field. Open-source software, in particular, reduces the need for developers to repeatedly check and fix foundational problems. As a result, it allows a more focused approach, where effort can be directed toward more important challenges and the creation of new functionalities. In this way, open source does not merely support development – it also actively accelerates it by allowing individuals and organizations to build upon existing knowledge rather than starting from first principles each time.

PART B — Linux Footprint

1. Installation Method

In the most modern **Linux** system, compiling MySQL from source is normally not required until someone is working on the internal development of the database engine. For regular usage, installation is handled through package management systems.

On Debian-based systems such as **Ubuntu,** MySQL is installed using the APT package manager. The commonly used commands are:

- *sudo apt update*
- *sudo apt install mysql-server*

By using a package manager we simplify the process significantly. It automatically resolves and installs all required dependencies, including essential system libraries, which reduces manual efforts and avoids configuration issues that may arise during source compilation.

2. Key Directories and Filesystem Integration

After installation, MySQL integrates into the Linux filesystem following standard directory conventions.

- **Binaries (Executables):**

  The MySQL client tool is located at */usr/bin/mysql*, which is used to interact with the database.

  The main server daemon, responsible for handling database operations, is found at */usr/sbin/mysqld*.

- **Configuration Files:**

```
varun11410@Varun11410:~$ ls -l /usr/bin/mysql
-rwxr-xr-x 1 root root 6969816 Jan 22 18:14 /usr/bin/mysql
varun11410@Varun11410:~$ ls -l /etc/mysql/my.cnf
lrwxrwxrwx 1 root root 24 Mar 29 14:05 /etc/mysql/my.cnf -> /etc/alternatives/my.cnf
varun11410@Varun11410:~$
```

  The primary configuration file is usually located at /etc/mysql/my.cnf (in some cases /etc/my.cnf).

This file is used to define system-level settings such as port number, memory allocation, and connection limits.

- **Log Files:**
  Logs are typically stored in /var/log/mysql/error.log.

  This file is useful for monitoring system behavior and diagnosing errors or crashes.

- **Data Directory:**
  The actual database files, including tables and records, are stored in /var/lib/mysql.

This directory contains all persistent data used by the database.

3. User and Group (Security Perspective)

During installation, MySQL automatically creates a dedicated system for user and group named mysql.

This is important for security purpose. The MySQL service does not run as the root user. If it did, any vulnerability in the database system could potentially give an attacker full control over the operating system. Instead, by running under a restricted user account, the system follows the principle of least privilege.

```
varun11410@Varun11410:~$ sudo ls -ld /var/lib/mysql
drwx------ 7 mysql mysql 4096 Mar 29 14:05 /var/lib/mysql
varun11410@Varun11410:~$ sudo ls -ld /var/log/mysql
drwxr-x--- 2 mysql adm 4096 Mar 29 14:05 /var/log/mysql
varun11410@Varun11410:~$
```

In practical terms, this means that even if the database is compromised, the damage is limited to what the mysql user can access, rather than affecting the entire system.

4. Service Management

MySQL operates as a background service (daemon) and is managed using systemd.

The following commands are used to control the service:0

- **Start the service:**
  sudo systemctl start mysql
- **Stop the service:**
  sudo systemctl stop mysql
- **Restart the service:**
  sudo systemctl restart mysql
- **Check service status:**
  sudo systemctl status mysql

```
varun11410@Varun11410:~$ sudo service mysql status
● mysql.service - MySQL Community Server
     Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
     Active: active (running) since Sun 2026-03-29 14:05:24 IST; 46s ago
    Process: 1598 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
   Main PID: 1606 (mysqld)
     Status: "Server is operational"
      Tasks: 38 (limit: 9306)
     Memory: 365.1M
        CPU: 957ms
     CGroup: /system.slice/mysql.service
             └─1606 /usr/sbin/mysqld

Mar 29 14:05:23 Varun11410 systemd[1]: Starting MySQL Community Server...
Mar 29 14:05:24 Varun11410 systemd[1]: Started MySQL Community Server.
```

The status command is particularly useful, as it shows whether the service is active along with recent log messages.

5. Update Model

MySQL updates are delivered through the same package management system used for installation. When security vulnerabilities are identified, developers fix them in the source code and release updated versions.

These updates are then packaged and distributed by Linux distribution maintainers (for example, Canonical in the case of Ubuntu) through official repositories.

To apply updates on a system, the following commands are used:

- *sudo apt update*
- *sudo apt upgrade*

This approach ensures that the system remains up to date with the latest security patches without requiring manual reinstallation or configuration.

PART C — The FOSS Ecosystem

Open-Source Dependencies

MySQL works as a part of a much larger open-source software stack and depends on several underlying tools and libraries for its functioning. It is primarily written in C and C++, so building it from source requires compilers such as GCC or Clang. In addition, build systems like CMake are used to manage the compilation process across platforms.

For secure communication, MySQL relies on OpenSSL, which enables encrypted data transfer between the client and server. It also depends on low-level system libraries such as glibc, which provide essential functionalities like memory management and interaction with the operating system kernel. These dependencies from the foundational layer that allows MySQL to function efficiently on Linux systems.

Role in the LAMP Stack and Web Ecosystem

MySQl is a core part of the widely used LAMP stack, where it represents the letter "M" – database layer. This stack played a major role in the development of early dynamics web applications.

In a typical setup, Linux is used as operating system, Apache handles the incoming web requests and a scripting language like Python or PHP is used to process the application logic. MySQL is responsible for storing and retrieving structured data, such as user information, transaction records or application outputs.

In a real life scenario, let's say a web-based system is designed to process agricultural data, the web server manages the interface, the application layer handle the processing (such as analysis or prediction), and MySQL stores results, user data, and past records. In this way, it acts as the persistent storage layer that supports the overall application.

Forks, Spin-offs, and Related Projects

The open-source nature of MySQL has led to the development of several derivative projects. One of the most famous examples is MariaDB. It was created after the concerns arose regarding the acquisition of MySQL by Oracle Corporation. The original creator, Michael Widenius, started this fork to ensure that a fully open and community-driven version of the database would continue to exist.

Another unique project is Percona Server, which focuses on performance optimization, especially for enterprise-level workloads. These projects are largely compatible with MySQL and are often used as alternatives.

Such forks demonstrate an important characteristic of open-source ecosystems i.e. if a community disagrees with the direction of a project, it has the ability to modify the source code and develop an independent version.

Community and Governance

The governance of MySQL is a unique combination of corporate control and community contribution.

- **Corporate Governance:** The MySQL trademark and the primary codebase are governed by Oracle Corporation. Oracle employs the core engineers who approve the pull requests, manage releases, and dictate the official roadmap.
- **The Grassroots Community:** Even there is corporate ownership, a massive global community surrounds the tool. Developers interact through the official MySQL forums, global Slack and Discord channels, and massive conferences like Oracle OpenWorld.
- **Bug Reporting:** The open-source community actively uses the public MySQL bug tracker. Independent developers from around the world constantly test the database, find possible vulnerabilities in them and submit detailed bug reports and patches, effectively providing Oracle with millions of dollars' worth of free quality assurance.

PART D — Open Source vs Proprietary

Dimension	Open Source (MySQL)	Proprietary Alternative (MS SQL Server)
Cost	Community edition(GPLv2) are free of cost. We only need to pay if we embed it in a closed-source product.	Cost is extremely high. It is calculated per CPU core or per user, might cost very much just for a standard enterprise deployment.
Security	The source code is publicly available, so developers around the world can review and identify issues, making it highly transparent.	The source code is closed, so only the company's internal team can review and fix vulnerabilities.
Support and reliability	Mostly supported by a global community through forums, documentation, and discussions, with optional paid support available.	Comes with official support from the company, including service-level agreements (SLAs) and 24/7 technical assistance.
Freedom to modify	Users have the flexibility to modify, customize, and even redistribute the software according to their needs.	Users cannot access or modify the source code, and must follow strict licensing terms.
Community vs corporate control	Development is influenced by the community, and if needed, the project can be forked (for example, MariaDB).	Fully controlled by the vendor, including updates, features, and long-term direction.

Conclusion

Would I recommend MySQL for real-world deployment?

Well according to me whether MySQL can be recommended for real-world deployment, the answer largely depends on scale and requirements of the project. For majority of the modern web application project, MySQL is the best and efficient choice. It offers reliable relational data management without any licensing cost, which helps the developers working within limited budgets. For example for building a system for processing image data or detecting plant diseases, MySQL integrates smoothly with backend technologies like Java or Python and benefits from extensive community support. Whereas, for bigger projects which have high-risk environment – where even a small downtime can lead to significant losses – Proprietary Alternatives like MS SQL Server might be the more efficient solution due to their guaranteed support and service-level agreements.

Would I contribute to it?

I absolutely would like to contribute to the MySQL ecosystem, but I will not directly start contributing to its core C and C++ engine. This is the best part of open source that contribution can take various forms. As my main focused is on web applications and system integrations, I would contribute by building and sharing improved wrapper libraries, writing clearer documentation for edge-case database migrations or by submitting detailed bug reports when the database interacts unexpectedly with newer machine learning frameworks. I believe contributing to such a foundational tool is not just an act of community service; it is a critical step in transitioning from a consumer of software to a builder of the modern web ecosystem.

Shell Scripts

Script 1: System Identity Report

Code-

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Varun
# Course: Open Source Software
# Purpose: Displays system information and OS license details.

# --- Variables ---
STUDENT_NAME="Varun"
SOFTWARE_CHOICE="MySQL"

# --- System Info Gathering ---
# uname -r fetches the exact Linux kernel version
KERNEL=$(uname -r)
# whoami fetches the currently logged-in user
USER_NAME=$(whoami)
# $HOME is an environment variable storing the user's home directory
HOME_DIR=$HOME
# uptime -p shows how long the system has been running in a pretty format
UPTIME=$(uptime -p)
# date formats the current time and day
CURRENT_DATE=$(date +"%A, %d %B %Y %T")
# cat reads the os-release file, grep finds the name, and cut extracts the exact string
DISTRO=$(cat /etc/os-release | grep "^PRETTY_NAME" | cut -d "" -f 2)
LICENSE_MSG="This OS is distributed under the GNU GPL and other open-source
licenses."

# --- Display Output ---
echo "========================================="
echo "         The Open Source Audit         "
echo "========================================="
echo "Student : $STUDENT_NAME"
echo "Software: $SOFTWARE_CHOICE"
echo "-----------------------------------------"
echo "Distro  : $DISTRO"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Home Dir: $HOME_DIR"
echo "Uptime  : $UPTIME"
echo "Date    : $CURRENT_DATE"
echo "-----------------------------------------"
echo "License : $LICENSE_MSG"
echo "========================================="
```

Output-

```
varun11410@Varun11410:~$ nano script1.sh
varun11410@Varun11410:~$ chmod +x script1.sh
varun11410@Varun11410:~$ ./script1.sh
========================================
          The Open Source Audit
========================================
Student : Varun
Software: MySQL
----------------------------------------
Distro  : Ubuntu 22.04.5 LTS
Kernel  : 6.6.87.2-microsoft-standard-WSL2
User    : varun11410
Home Dir: /home/varun11410
Uptime  : up 1 minute
Date    : Monday, 30 March 2026 06:44:34
----------------------------------------
License : This OS is distributed under the GNU GPL and other open-source licenses.
========================================
```

Script 1 Analysis:

This script provides a dynamic welcome dashboard displaying system identity information. It relies heavily on variables to store static strings (like the student name) and command substitution ($()) to dynamically execute Linux commands (such as uname -r, whoami, and uptime -p) and capture their output into variables. I also utilized a pipeline involving cat, grep, and cut to cleanly extract the human-readable Linux distribution name directly from the /etc/os-release configuration file. Finally, echo statements were used with basic output formatting to present the data in a clean, readable UI.

Script 2: FOSS Package Inspector
Code-

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Varun
# Course: Open Source Software
# Purpose: Checks if MySQL is installed and prints its FOSS philosophy.

# The package name for MySQL on Ubuntu
PACKAGE="mysql-server"

echo "Inspecting Package: $PACKAGE..."
echo "----------------------------------------"

# --- Check if package is installed ---
# dpkg -l lists packages, grep -q searches silently for the exact package name
if dpkg -l | grep -q "^ii  $PACKAGE "; then
   echo "[SUCCESS] $PACKAGE is installed on this system."
   echo "--- Package Details ---"
   # Extract version details using dpkg-query
   dpkg-query -W -f='Version: ${Version}\n' $PACKAGE
   echo "----------------------------------------"
else
   echo "[WARNING] $PACKAGE is NOT installed."
   echo "Please install it using 'sudo apt install $PACKAGE'."
   # Exit the script with an error code if not found
   exit 1
fi

# --- Philosophy Note ---
# Case statement to match the package name and print its purpose
echo "--- FOSS Philosophy Note ---"
case $PACKAGE in
   "apache2" | "httpd")
      echo "Apache: The web server that built the open internet."
      ;;
   "mysql-server" | "mysql")
      echo "MySQL: Open source at the heart of millions of web applications."
      ;;
   "firefox")
```

```
        echo "Firefox: A nonprofit fighting for an open and private web."
        ;;
    "vlc")
        echo "VLC: Plays anything - built by students in Paris."
        ;;
    *)
        echo "$PACKAGE: A valuable contribution to the open-source ecosystem."
        ;;
esac
echo "========================================="
```

Output-

```
varun11410@Varun11410:~$ nano script2.sh
varun11410@Varun11410:~$ chmod +x script2.sh
varun11410@Varun11410:~$ ./script2.sh
Inspecting Package: mysql-server...
-----------------------------------------
[SUCCESS] mysql-server is installed on this system.
--- Package Details ---
Version: 8.0.45-0ubuntu0.22.04.1
-----------------------------------------
--- FOSS Philosophy Note ---
MySQL: Open source at the heart of millions of web applications.
=========================================
```

Script 2 Analysis:

This script inspects the local system to verify if a chosen open-source package is installed. It utilizes an if-then-else conditional structure combined with a pipeline (dpkg -l | grep -q) to silently check the installation status of the software. If installed, it uses dpkg-query to extract and print the specific version. The script concludes with a case statement, a highly efficient flow-control concept that acts as a switch, identifying the package name (mysql-server) and printing a predefined, one-line philosophical statement regarding its role in the open-source ecosystem.

Script 3: Disk and Permission Auditor.
Code-

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Varun
# Course: Open Source Software
# Purpose: Audits critical system directories and MySQL configs using loops.

# Array containing standard Linux directories plus the MySQL config directory
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/etc/mysql")

echo "========================================="
echo "      Directory Audit Report          "
echo "========================================="

# A for loop to iterate through every item in the DIRS array
for DIR in "${DIRS[@]}"; do
   # Check if the directory actually exists on the system
   if [ -d "$DIR" ]; then
      # Use ls -ld piped to awk to extract just the permissions ($1), owner ($3), and group
($4)
      PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')

      # Use du -sh to get the human-readable size.
      # 2>/dev/null hides 'Permission denied' errors for files we can't read as a normal
user.
      SIZE=$(sudo du -sh "$DIR" 2>/dev/null | awk '{print $1}')

      echo "Directory : $DIR"
      echo "Permissions: $PERMS"
      echo "Total Size : $SIZE"
      echo "----------------------------------------"
   else
      # Fallback if the directory is missing
      echo "[WARNING] Directory $DIR does not exist on this system."
      echo "----------------------------------------"
   fi
done

echo "Audit Complete."
```

echo "==========="

Output-

```
varun11410@Varun11410:~$ nano script3.sh
varun11410@Varun11410:~$ chmod +x script3.sh
varun11410@Varun11410:~$ ./script3.sh
========================================
       Directory Audit Report
========================================
[sudo] password for varun11410:
Directory : /etc
Permissions: drwxr-xr-x root root
Total Size : 4.2M
----------------------------------------
Directory : /var/log
Permissions: drwxrwxr-x root syslog
Total Size : 34M
----------------------------------------
Directory : /home
Permissions: drwxr-xr-x root root
Total Size : 56K
----------------------------------------
Directory : /usr/bin
Permissions: drwxr-xr-x root root
Total Size : 204M
----------------------------------------
Directory : /tmp
Permissions: drwxrwxrwt root root
Total Size : 40K
----------------------------------------
Directory : /etc/mysql
Permissions: drwxr-xr-x root root
Total Size : 44K
----------------------------------------
Audit Complete.
========================================
```

Script 3 Analysis:

This script automates a system audit by iterating through an array of critical Linux directories, including the MySQL configuration directory (/etc/mysql). It utilizes a for loop to process each path sequentially. Inside the loop, an if-then statement verifies the directory's existence (-d). The script demonstrates advanced text processing by piping the output of ls -ld into awk to extract specific columns representing file permissions and ownership. Similarly, it calculates directory sizes using du -sh, redirecting standard error (2>/dev/null) to maintain a clean terminal output. This script highlights how automation can rapidly verify the security posture of database configuration files.

Script 4: Log File Analyzer.
Code-

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Varun
# Course: Open Source Software
# Usage: ./script4.sh <path_to_log> <keyword>

# Command-line arguments
LOGFILE=$1
# Default keyword is 'error' if the user doesn't provide a second argument
KEYWORD=${2:-"error"}
COUNT=0

echo "========================================="
echo "        Log File Analyzer            "
echo "========================================="

# Do-while style retry loop: checks if file is missing or empty (-s)
while true; do
   if [ -z "$LOGFILE" ] || [ ! -s "$LOGFILE" ]; then
      echo "[!] Error: File '$LOGFILE' is missing or empty."
      read -p "Please enter a valid log file path (e.g., /var/log/dpkg.log): " LOGFILE
   else
      break # Exit the infinite loop once a valid file is provided
   fi
done

echo "Analyzing $LOGFILE for keyword: '$KEYWORD'..."

# while-read loop to process the file line by line
while IFS= read -r LINE; do
   # Check if the current line contains the keyword (case-insensitive)
   if echo "$LINE" | grep -iq "$KEYWORD"; then
      # Increment the counter
      COUNT=$((COUNT + 1))
   fi
done < "$LOGFILE"

echo "----------------------------------------"
```

```
echo "RESULT: Keyword '$KEYWORD' found $COUNT times."
echo "----------------------------------------"
echo "Last 5 matching lines:"
# Combine grep and tail to fulfill the final assignment requirement
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
echo "========================================"
```

Output-

```
varun11410@Varun11410:~$ nano script4.sh
varun11410@Varun11410:~$ chmod +x script4.sh
varun11410@Varun11410:~$ ./script4.sh
========================================
          Log File Analyzer
========================================
[!] Error: File '' is missing or empty.
Please enter a valid log file path (e.g., /var/log/dpkg.log): /var/log/dpkg.log
Analyzing /var/log/dpkg.log for keyword: 'error'...
----------------------------------------
RESULT: Keyword 'error' found 14 times.
----------------------------------------
Last 5 matching lines:
2026-03-09 14:17:34 status unpacked liberror-perl:all 0.17029-1
2026-03-09 14:18:10 configure liberror-perl:all 0.17029-1 <none>
2026-03-09 14:18:10 status unpacked liberror-perl:all 0.17029-1
2026-03-09 14:18:10 status half-configured liberror-perl:all 0.17029-1
2026-03-09 14:18:10 status installed liberror-perl:all 0.17029-1
========================================
varun11410@Varun11410:~$ |
```

Script 4 Analysis:

This script serves as a diagnostic tool, processing server log files line-by-line to identify and count specific events like system errors. It accepts command-line arguments ($1 and $2), implementing default parameter substitution if the user omits the search keyword. To fulfill the retry requirement, it uses an infinite while true loop combined with a break condition to simulate a do-while loop, continuously prompting the user until a valid, non-empty log file (-s) is provided. The core logic relies on a while read loop combined with grep to parse the text natively, and it concludes by combining grep and tail to present the five most recent occurrences of the specified keyword.

Script 5: The Open Source Manifesto Generator.
Code-

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Varun
# Course: Open Source Software
# Purpose: Interactively generates a personalized manifesto and saves it to a text file.

# --- Concept Demonstration ---
# The rubric requires demonstrating an alias via a comment.
# An alias creates a custom shortcut command. For example:
# alias read_manifesto="cat manifesto_$(whoami).txt"

echo "========================================="
echo "   Open Source Manifesto Generator      "
echo "========================================="
echo "Answer three questions to generate your manifesto."
echo

# read -p prompts the user and stores their answer in a variable
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# Get the current date formatted as 'Day Month Year'
DATE=$(date +'%d %B %Y')
# Create a dynamic filename based on the logged-in user
OUTPUT="manifesto_$(whoami).txt"

# --- File Generation ---
# Using a single > overwrites/creates the file.
# Using >> appends to the file.
echo "========================================" > "$OUTPUT"
echo "       MY OPEN SOURCE MANIFESTO        " >> "$OUTPUT"
echo "       Date: $DATE                     " >> "$OUTPUT"
echo "========================================" >> "$OUTPUT"
echo "I believe that software should empower, not restrict. Every day, I rely on tools like
$TOOL to build and learn, standing on the shoulders of the open-source giants who came
before me. To me, digital freedom ultimately means $FREEDOM. Because of the
community that shared their code with me, I am committed to paying it forward. In the
```

future, I pledge to build $BUILD and share it freely with the world, ensuring the next generation of developers has the same opportunities I did." >> "$OUTPUT"

```
echo
echo "[SUCCESS] Manifesto generated and saved to $OUTPUT"
echo "----------------------------------------"
echo "Here is what you wrote:"
echo

# Display the final text file to the user
cat "$OUTPUT"
echo "==========================================="
```

Output-

```
varun11410@Varun11410:~$ nano script5.sh
varun11410@Varun11410:~$ chmod +x script5.sh
varun11410@Varun11410:~$ ./script5.sh
========================================
   Open Source Manifesto Generator
========================================
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: Python
2. In one word, what does 'freedom' mean to you? Liberty
3. Name one thing you would build and share freely: accessible AI models

[SUCCESS] Manifesto generated and saved to manifesto_varun11410.txt
----------------------------------------
Here is what you wrote:

========================================
       MY OPEN SOURCE MANIFESTO
       Date: 30 March 2026
========================================
I believe that software should empower, not restrict. Every day, I rely on tools like Python to build and learn,
 standing on the shoulders of the open-source giants who came before me. To me, digital freedom ultimately means
 Liberty. Because of the community that shared their code with me, I am committed to paying it forward. In the
future, I pledge to build accessible AI models and share it freely with the world, ensuring the next generation
of developers has the same opportunities I did.
========================================
varun11410@Varun11410:~$
```

Script 5 Analysis:

This script serves as an interactive text generator. It utilizes the read -p command to pause execution, prompt the user with questions, and store their standard input into designated variables. To handle file I/O operations, it uses string concatenation alongside output redirection (> to create the file and >> to append subsequent lines of the composed paragraph). Additionally, the script dynamically names the output file using command substitution (whoami) and injects the current date utilizing the date command. A demonstration of shell aliases is included as a header comment to satisfy the conceptual requirement.